Market Maker: Design Document

Ulysses Butler

Fall 2026

Contents

1 Context **1**
1.1 The simulator . . . 1
1.2 Trading strategy . . . 2

2 Application **3**
2.1 Evaluation . . . 3
2.2 Market Maker . . . 3
2.2.1 The book . . . 4
2.2.2 Pricing . . . 4
2.2.3 Risk . . . 4
2.2.4 Order management . . . 5
2.2.5 General . . . 5

Appendices **5**

Appendix A Trading strategy **5**
A.1 Fair value . . . 5
A.2 Credit . . . 6
A.3 Quotes . . . 6
A.4 When we requote . . . 6
A.5 Risk . . . 7

Appendix B Market Data API **7**

Appendix C Order API **8**

1 Context

1.1 The simulator

The simulator is a discrete-event simulation with an exchange at its heart. Market participants send the exchange requests to enter, replace, and cancel orders (using the API detailed in Appendix C). The exchange matches those orders and publishes a stream of market data describing everything that happens on its book, such as orders being added, executed, and canceled (using the format detailed in Appendix B). It also sends each participant responses about its own orders (also described in Appendix C).

The exchange, the network between it and our application, and the other market participants are all simulated in the same process, on a single thread. The other participants' behavior comes from a scripted scenario file (`sim/fixtures/*.scn`), which lists the orders they send over the course of our simulated day. Their orders reach the exchange immediately. Ours travel over a simulated network with a fixed one-way latency: by default, market data reaches us 5 µs after the exchange publishes it, and orders and their responses take 10 µs in each direction. This is why, for example, we only learn the order ref of an order we send a full round trip after sending it.

Our application sees the simulator only through three handlers.

- `OnMarketData`: a market data message has arrived.
- `OnExecutionReport`: the exchange has responded about one of our orders (for example, it was accepted or filled).
- `OnTimer`: a periodic timer has fired (the interval is set by the scenario).

Our firm is a market maker: we continuously quote a bid just below our estimate of the instrument's fair value and an offer just above it (summarized in Section 1.2). Most of our orders are sent from `OnMarketData`, when the market moves, and from `OnExecutionReport`, when one of our own orders is filled or acknowledged. Orders are sent by calling `OrderEntrySession::Send`, which may only be called from inside one of these handlers.

A run, therefore, looks like this: the simulator feeds the scenario's orders to the exchange, the exchange publishes market data, our application reacts by sending orders of its own, and the exchange responds to those, which our application reacts to in turn.

The simulator keeps two kinds of time. *Simulated time* is the scenario's schedule: when each participant acts, and when each message arrives, given the latencies above. It decides what happens in the market, and it is identical on every machine and every run. Simulated time stands still while one of our handlers runs, so how fast our code is can never change which orders we send or when (in simulated time) we send them. *Wall-clock time* is real elapsed time on the machine running the simulation, and it is what measures our application's performance. Simulated time should never be used to evaluate performance.

Wall-clock time at the network boundary is recorded by a *capture tap*. It records the wall-clock time at which each message reaches our application (just before the handler is called) and the time at which each of our orders leaves it (when `Send` is called), along with which message was being handled when the order was sent. The tap is part of the simulator, so our application cannot change what it records.

All messages are packed structs with fixed field offsets. Prices are fixed-point integers with four implied decimal places ($100.25 is `1002500`), and the tick size is $0.01. The simulator trades a single instrument.

1.2 Trading strategy

Our trading strategy is fixed: it was decided by another division, our application must reproduce it exactly, and `make verify` checks that it does. In short:

- Our estimate of fair value is the *microprice* of the best bid and offer, computed without our own orders.
- We quote one order on each side: a bid below fair value and an offer above it, each a *credit* away, rounded to the tick grid away from fair value.

- The credit widens with volatility, which is sampled on the timer, not on every message.
- After every market data message and every response from the exchange, we re-evaluate both sides and enter or replace an order if its price should change, with at most one request in flight per side.
- Four risk conditions limit or stop quoting: an invalid book, a gap in the market data, the position limit, and the loss limit.

The full specification, including every formula, parameter, and risk reaction, is in Appendix A.

2 Application

2.1 Evaluation

We measure the application's performance with the capture tap, which timestamps every message as it reaches our application (each market data tick, each response from the exchange, and each timer tick), outside of our own code. Not every tick results in a trade, though: most market data messages leave our quotes unchanged, and the timer and the exchange's responses do work of their own. To account for all of that work, each of our handlers records a wall-clock timestamp (using `sim::WallClockNow()`, the same clock the tap uses) when it finishes, together with a key identifying the message: its sequence number, or the time of a timer tick. Joining the two gives the time each handler took, from the tick's arrival to the end of our processing, and the latency report averages that over every message in the run. The log itself is kept cheap, so that it does not distort what it measures: records go into an array preallocated before the run starts, with no I/O and one clock read per handler. The code is in `app/evaluation/`, and the report is printed at the end of every run of `build/market_maker`.

2.2 Market Maker

The market maker is everything in `app/` outside `app/evaluation/`. It is organized into a few components, each owning one part of the problem:

- **The book** (`app/market_data/book.{h,cpp}`) rebuilds the order book from the market data feed and keeps the best bid and offer, excluding our own orders.
- **Pricing** (`app/strategy/pricing.h`) computes fair value, volatility, credit, and quote prices.
- **Risk** (`app/risk/risk.h`) implements the four risk conditions.
- **Order management** (`app/execution/`) tracks our resting orders and in-flight requests, assigns tokens, and sends orders.
- **The market maker** itself (`app/market_maker.{h,cpp}`) implements the three handlers and ties the components together.

All parameters are in `app/config.h`. The rest of this section describes the main design decisions in each component, and the reasoning behind them.

2.2.1 The book

Order lookup. Most market data messages identify an order only by its exchange order ref, so the book must find an order by ref on nearly every message. Rather than a hash map, the book stores orders in a flat array indexed directly by ref. The exchange assigns refs in increasing order, so the array is dense, and a lookup is a single indexed load: no hashing, no collisions, and no pointer chasing. A hash map lookup, by contrast, must hash the key and, with `std::unordered_map`, follow a pointer into a separately allocated node, which is often a cache miss. The array is allocated and zeroed once, at startup, for up to about a million refs (16 MB).

Price levels. The total size resting at each price is kept in a ladder: one preallocated slot per tick, per side, covering prices up to $2,000. Adding or removing size at a price is a single indexed update, with no tree to walk and no allocation when a new price level appears.

Top of book. The strategy reads the best bid and offer on every message. Instead of searching the book for them, the book keeps a small cached copy of the best price and size on each side, updated incrementally whenever an order at or better than the best changes. Reading the top of book is then a copy of four fields. This is deliberate denormalization: we keep a duplicate of exactly the data the hotpath needs, rather than recomputing it from the full book. When the best level empties, the next one is found by scanning the price level ladder from the previous best toward worse prices; levels near the top of the book are close together, so the scan is short.

Decoding. Each handler receives a message's raw bytes and copies them into the matching packed struct with `memcpy` (via `sim::Decode`), rather than casting the byte pointer to the struct type. This avoids misaligned access and undefined behavior, and the compiler reduces a small fixed-size copy to a few loads.

2.2.2 Pricing

Price arithmetic. Prices are 64-bit fixed-point integers throughout. The microprice is a single integer division, and rounding to the tick grid is integer arithmetic as well. The result is exact and identical on every machine and compiler, which is what allows `make verify` to compare our output exactly.

Credit computation. The credit depends on the volatility estimate and the mid price sampled on the timer. The application recomputes it from those inputs on every message, so that every quote is always priced off the latest volatility estimate.

does not need to happen every message

Buy and sell logic. Several calculations differ only slightly between the two sides: bids round down and asks round up, and the position limit is checked in opposite directions. Both sides can thus be handled by a single function that simply branches depending on which side it's given.

2.2.3 Risk

Risk checks. Our first principle is safety: mistakes can be expensive for the firm, so we always run a series of risk checks before sending out any quote (the checks themselves are part of the fixed strategy). Exceptions are free when not thrown, and once they are thrown, we are no longer in the hotpath, so latency is no longer critical. Thus, for every potential risk, we run the check, then throw

an exception if we need to abort the quote. Exceptions cannot be accidentally ignored in the same way a return code can. So, this is the safest approach, and one that won't incur a performance penalty on the hotpath.

Halting. When a sequence gap or the loss limit is detected, the application cancels every order and stops quoting. This happens at most once per run, so the handler that does it is marked `[[gnu::noinline]]`. Keeping it out of line keeps this rarely used code out of the quoting functions and their instruction-cache footprint.

2.2.4 Order management

Decision log. Every order is written to a decision log (`decisions.log`) before it is sent. If the process dies mid-send, the log still shows what we intended to have on the exchange, so our state can be reconstructed on restart. Logging after the send would leave a window in which an order is live on the exchange with no record of it.

Quote history. Every quote we send is also appended to an in-memory history. The history is kept for the whole run, because it is needed for end-of-day reconciliation against the exchange's records; the timer checks the new entries as they accumulate.

2.2.5 General

Cache behavior. The application does not run any warm-up work to keep its caches primed. It is small enough that its code and data fit comfortably in cache, and nearly every message already runs the book and pricing code.

Appendix A Trading strategy

The trading strategy is fixed: it was decided by another division, and the application must reproduce it exactly. Everything in this section is checked by `make verify`. The parameters are defined in `app/config.h`.

Parameter	Meaning	Value
`kQuoteShares` (Q)	Size of each quote	100 shares
`kMinCredit`	Minimum half-spread	\$0.01
`kCreditK` (k)	Volatility multiplier	2.0
`kPositionLimit`	Maximum absolute position	500 shares
`kLossLimit`	Maximum loss before quoting stops	\$100

A.1 Fair value

Our estimate of the instrument's value is the *microprice* of the best bid and offer:

$$\text{fairValue} = \frac{P_{\text{bid}} \cdot Q_{\text{ask}} + P_{\text{ask}} \cdot Q_{\text{bid}}}{Q_{\text{bid}} + Q_{\text{ask}}},$$

where P is the best price on each side and Q the total size resting at that price. Each side's price is weighted by the *opposite* side's size, so fair value leans toward the side that is more likely to trade next. It is computed in integer arithmetic, so the division rounds down to \$0.0001.

The best bid and offer exclude our own orders: our quotes must never move our own fair value. An order counts as ours once we have received the `Accepted` or `Replaced` response carrying its order ref, or once the feed shows it replacing one of our orders. Until then, it counts as someone else's liquidity.

A.2 Credit

The *credit* is the half-spread we require: how far from fair value we are willing to buy and sell. It widens when the market is volatile.

Volatility is estimated on the timer, not on every message. On each timer tick, if both sides of the book are present, we sample the mid price $m = \lfloor (P_{\text{bid}} + P_{\text{ask}})/2 \rfloor$ and update an exponentially weighted moving average of squared log returns:

$$r = \ln(m/m_{\text{prev}}), \qquad \sigma^2 \leftarrow \lambda\sigma^2 + (1-\lambda)\, r^2, \qquad \lambda = 0.94.$$

If a side of the book is empty, the sample is skipped and the previous values are kept. The credit is then

$$\text{credit} = \max\big(\texttt{kMinCredit},\ \lceil k \cdot \sigma \cdot m \rceil\big),$$

rounded up to \$0.0001. Before there are two samples, $\sigma = 0$, so the credit is `kMinCredit`.

Sampling on the timer is a modeling decision, not a performance one. Returns between consecutive messages are dominated by prices bouncing between the bid and the offer, and an estimator updated per message would change its time scale whenever the message rate changes. Credit should reflect volatility over a fixed horizon, not over whatever number of messages happened to arrive, so it is sampled at a fixed interval.

A.3 Quotes

We keep one order on each side, of Q shares, as a `Day` order:

$$\text{bid} = \left\lfloor \frac{\text{fairValue} - \text{credit}}{\text{tick}} \right\rfloor \cdot \text{tick}, \qquad \text{ask} = \left\lceil \frac{\text{fairValue} + \text{credit}}{\text{tick}} \right\rceil \cdot \text{tick}.$$

Rounding is always away from fair value, so it never narrows the spread.

A.4 When we requote

After *every* market data message and *every* response from the exchange, we re-evaluate both sides, bid first, then ask. We do not requote on timer ticks, even though the credit may have changed. For each side:

- if we have no order resting, we enter one at the desired price;
- if our resting order's price differs from the desired price, we replace it at the desired price, for Q shares (this also restores full size after a partial fill);
- otherwise, we do nothing.

Each side has at most one request in flight. While a request for a side (an `EnterOrder`, `ReplaceOrder`, or `CancelOrder`) awaits its response, we send nothing else for that side. When a fill completely removes a resting order, the side has no order, and we re-enter it while handling that fill. After a partial fill, the remainder stays on the book until the next requote.

Tokens are assigned sequentially from 1: each `EnterOrder` and `ReplaceOrder` takes the next one.

A.5 Risk

Four conditions stop or limit quoting:

Condition	Detected	Reaction
Book invalid	A side of the book (excluding our orders) is empty, or the book is crossed.	Skip this requote; our resting orders stay.
Sequence gap	A market data sequence number was skipped.	Cancel all orders; stop quoting for the rest of the run.
Position limit	A new buy of Q shares could take our position above $+500$, or a new sell below -500.	Don't send on that side; the other side still quotes. A resting order on the blocked side is left alone.
Loss limit	On a timer tick, our profit and loss is below $-\$100$.	Cancel all orders; stop quoting for the rest of the run.

Profit and loss is marked to the mid price sampled on the timer: $\text{cash} + \text{position} \cdot m$, where cash changes by the price times the shares of every fill. It is not checked until the first mid price has been sampled.

Cancel all cancels the resting order on each side, and also any order whose `EnterOrder` or `ReplaceOrder` is still in flight, bid first, then ask.

Appendix B Market Data API

The market data feed is modeled on NASDAQ ITCH. It is order-by-order: rather than publishing price levels, it reports every individual order that is added to or removed from the book. Every order is displayed, so the feed includes the application's own orders, unmarked.

Every message starts with a header carrying the message type, the instrument, a **sequence number** (starting at 1 and increasing by 1 per message, so a gap means a message was missed), and the time the exchange processed the event.

`AddOrder (A)`
: *Fields:* order ref, side, shares, price.
A new order is resting on the book. The order ref is the exchange's identifier for it; later messages about this order carry only the ref.

`OrderExecuted (E)`
: *Fields:* order ref, executed shares, match number.
Part or all of a resting order traded. This is the feed's trade print.

`OrderCancel (X)`
: *Fields:* order ref, canceled shares.
A resting order was reduced in size, but is still on the book.

`OrderDelete (D)`
: *Fields:* order ref.
A resting order was removed from the book entirely.

`OrderReplace (U)`

Fields: original order ref, new order ref, shares, price.
A resting order was replaced by a new one at a new price and/or size. The new order has a new ref and loses its place in the queue.

Appendix C Order API

The order entry protocol is modeled on NASDAQ OUCH. The application identifies its own orders by a **token** that it chooses, which must be unique for the whole session. The exchange's order ref is only known once the exchange's response arrives, a full round trip later.

Requests (application to exchange).

`EnterOrder (O)`

Fields: token, side, shares, instrument, price, time in force (`Day` or `IOC`).
Places a new order. A `Day` order rests on the book until it is filled or canceled; an `IOC` (immediate-or-cancel) order trades what it can immediately and cancels the rest. A reused token is silently ignored, so a retransmission is safe.

`ReplaceOrder (U)`

Fields: existing token, replacement token, shares, price.
Replaces a live order with a new one at a new price and/or size. The replacement takes the new token and loses its place in the queue.

`CancelOrder (X)`

Fields: token, shares.
Reduces a live order to `shares` remaining; 0 cancels it entirely. Requests for unknown or already-dead tokens are ignored.

Responses (exchange to application). Each response carries a header with its type, a sequence number (per session, starting at 1), and the time the exchange processed the event.

`Accepted (A)`

Fields: token, side, shares, instrument, price, time in force, order ref.
The order was accepted. This is the first time the application learns its order ref.

`Replaced (U)`

Fields: replacement token, previous token, side, shares, price, order ref.
The replacement is live, with a new order ref.

`Executed (E)`

Fields: token, executed shares, price, match number, liquidity flag.
One of our orders traded. The liquidity flag says whether our order was resting (added liquidity) or was the aggressor (removed liquidity).

`Rejected (J)`

Fields: token, reason.
The request was refused: an invalid price or size, an unknown instrument, or a replace of an order that is no longer live.

`Canceled (C)`

Fields: token, shares canceled, reason.

Some or all of an order was canceled, either at our request or because it was the unfilled remainder of an `IOC` order.